Week 8 : Hash Table Objectives

Upon completion of this Binary tree and heaps unit, students will be able to understand and explain the following sub topics :

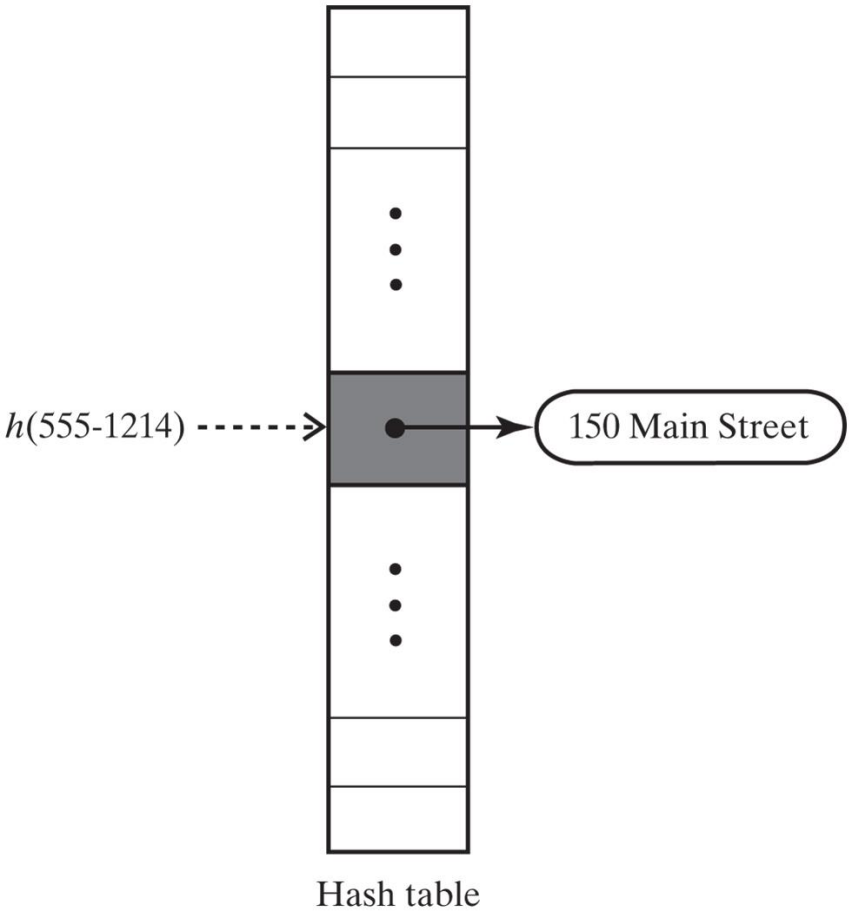
- What Is Hashing?
- Hash table and Functions
- Computing Hash Codes
- Compressing a Hash Code into an Index for the Hash Table
- Table of Contents Resolving Collisions
- Open Addressing with Linear Probing
- Hashing as a Dictionary Implementation
- The Efficiency of Hashing
- The Load Factor
- The Cost of Open Addressing
- Comparing Schemes for Collision Resolution
- A Dictionary Implementation That Uses Hashing

Hashing

- A technique that determines an index into a table using only an entry's search key
- Hash function
  - Takes a search key and produces the integer index of an element in the hash table
  - Search key is mapped, or hashed, to the index

Hash Table

- FIGURE 2 its hash table



© 2019 Pearson Education, Inc.

Ideal Hashing

- Simple algorithms for the dictionary operations that add and retrieve

```
Algorithm add(key, value)
index = h(key)
hashTable[index] = value
```

```
Algorithm getValue(key)
index = h(key)
return hashTable[index]
```

Typical Hashing

- Typical hash functions perform two steps:
  - Convert search key to an integer
    - Called the hash code.
  - Compress hash code into the range of indices for hash table.

```
Algorithm getHashIndex(phoneNumber)
// Returns an index to an array of tableSize elements.
i = last four digits of phoneNumber
return i % tableSize
```

Typical Hashing

- Most hash functions are not perfect,
  - Can allow more than one search key to map into a single index
  - Causes a collision in the hash table
- Consider `tableSize = 101`
- `getHashIndex(555-1214) = 52`
- `getHashIndex(555-8132) = 52` *also!!!*

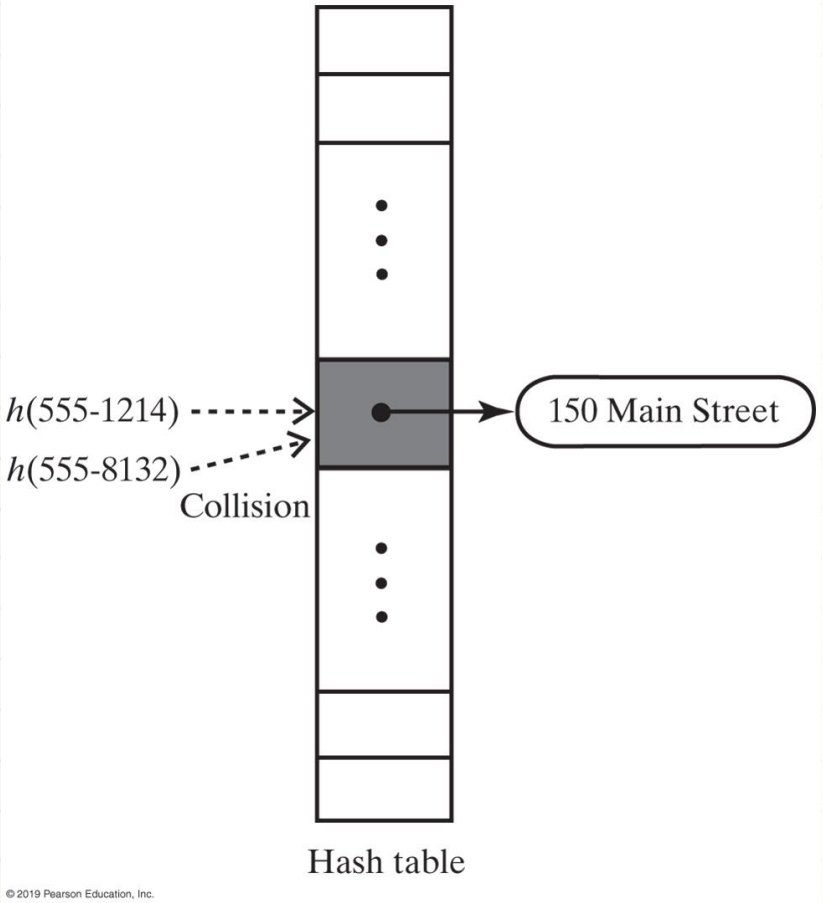


FIGURE 22-2 A collision caused by the hash function *h*

Hash Functions

- A good hash function should
  - Minimize collisions
  - Be fast to compute
- To reduce the chance of a collision
  - Choose a hash function that distributes entries uniformly throughout hash table.

Computing Hash Codes

- Java's base class **`Object`** has a method **`hashCode`** that returns an integer hash code
  - A class should define its own version of **`hashCode`**
- A hash code for a string
  - Using a character's Unicode integer is common
  - Better approach:
    - Multiply Unicode value of each character by factor based on character's position,
    - Then sum values

Computing Hash Codes

- Hash code for a string example:

$$u_0 g^{n-1} + u_1 g^{n-2} + \ldots + u^{n-2} g + u_{n-1}$$

- Java code to do this:

```
int hash = 0;
int n = s.length();
for (int i = 0; i < n; i++)
   hash = g * hash + s.charAt(i);
```

Hash Code for a Primitive type

- If data type is **`int`**,
  - Use the key itself
- For **`byte, short, char`**:
  - Cast as **`int`**
- Other primitive types
  - Manipulate internal binary representations

Compressing a Hash Code

- Common way to scale an integer
  - Use Java mod operator **%**: **`code % n`**
- Best to use an odd number for n
- Prime numbers often give good distribution of hash values

Compressing a Hash Code

- Hash function for the ADT dictionary

```
private int getHashIndex(K key)
{
   int hashIndex = key.hashCode() % hashTable.length;
   if (hashIndex < 0)
      hashIndex = hashIndex + hashTable.length;

   return hashIndex;
} // end getHashIndex
```

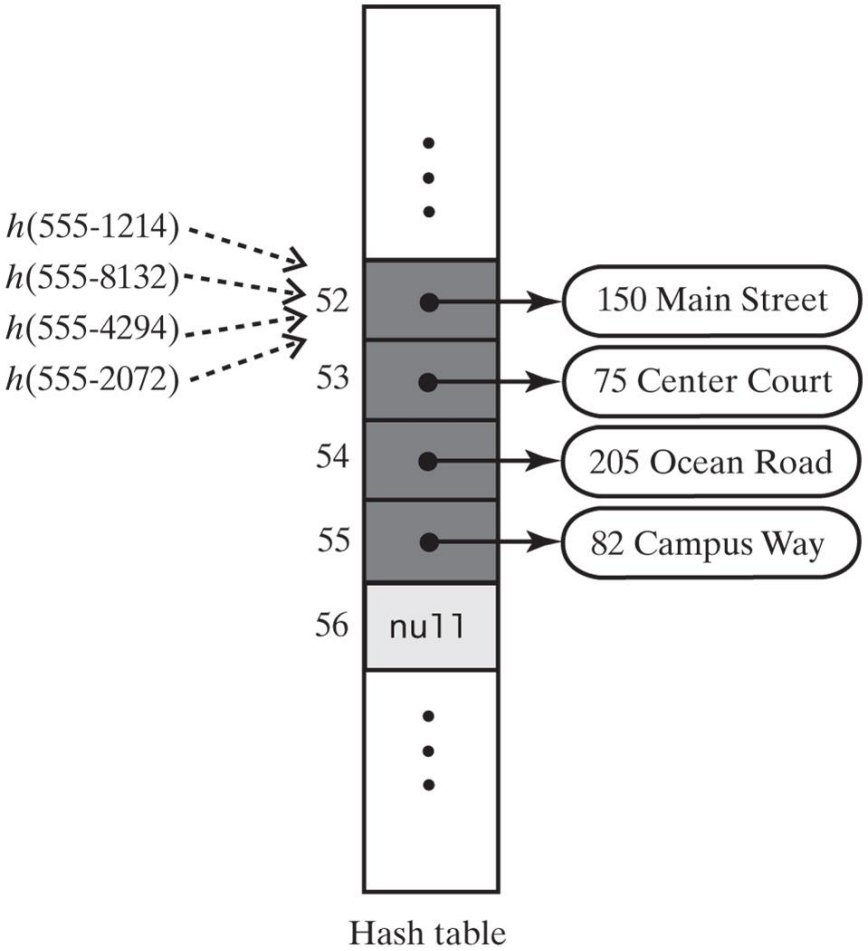
Resolving Collisions

- Collision:
  - Hash function maps search key into a location in hash table already in use
- Two choices:
  - Use another location in the hash table
  - Change the structure of the hash table so that each array location can represent more than one value

Resolving Collisions

- **Linear probing**
  - Resolves a collision during hashing by examining consecutive locations in hash table
  - Beginning at original hash index
  - Find the next available one
- Table locations checked make up probe sequence
- If probe sequence reaches end of table, go to beginning of table (circular hash table)

Linear Probing

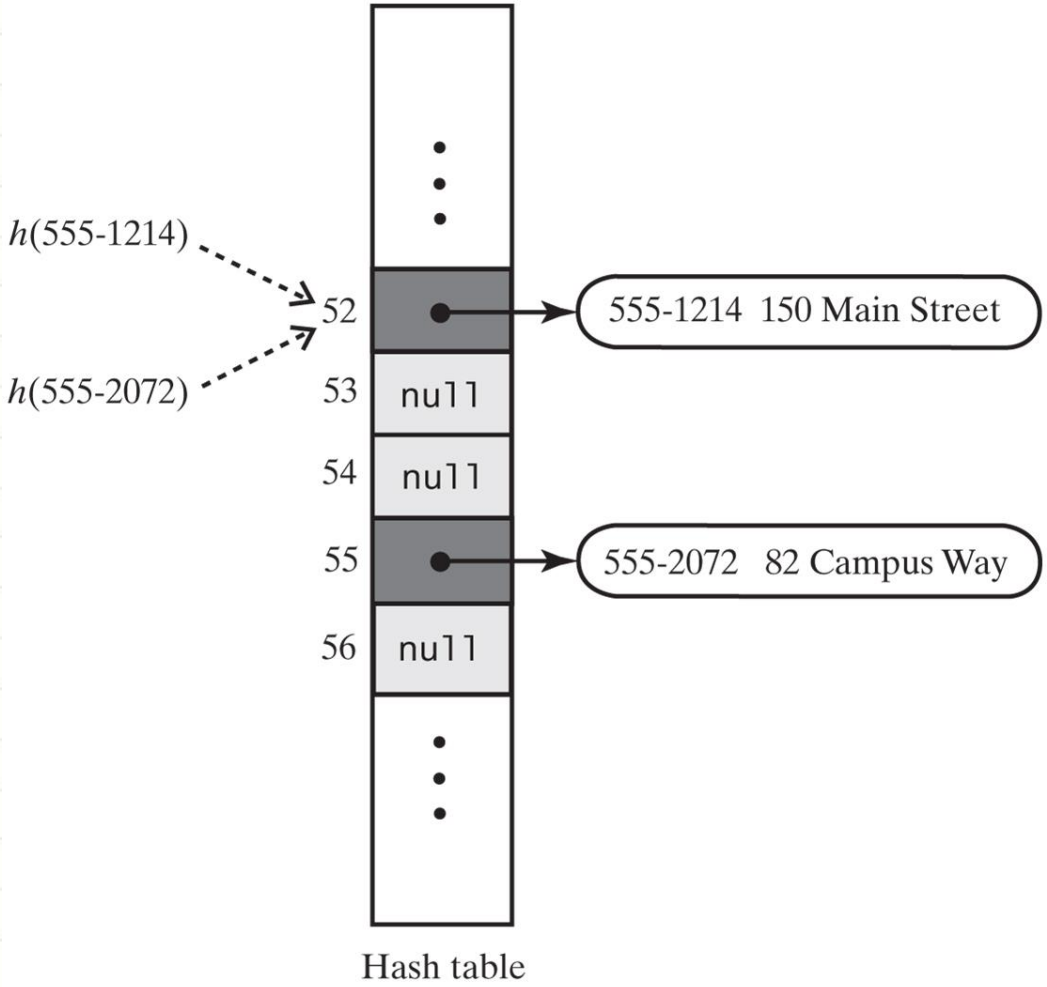


© 2019 Pearson Education, Inc.

- The effect of linear probing after adding four entries whose search keys hash to the same index

Linear Probing

- FIGURE remove entries



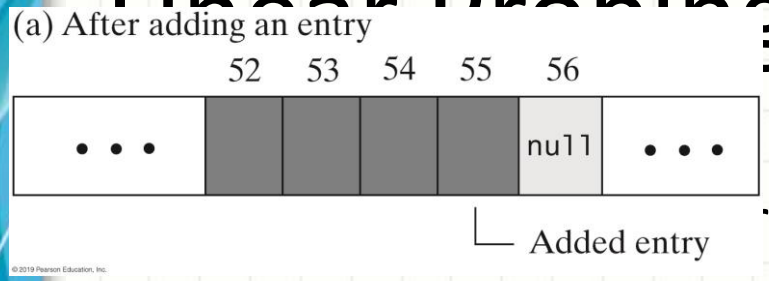
© 2019 Pearson Education, Inc.

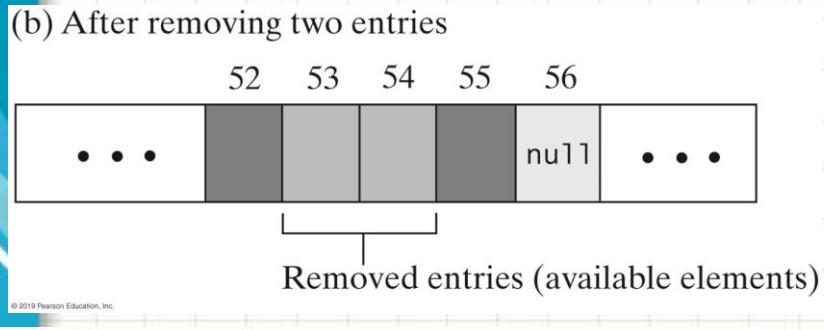
Resolving Collisions

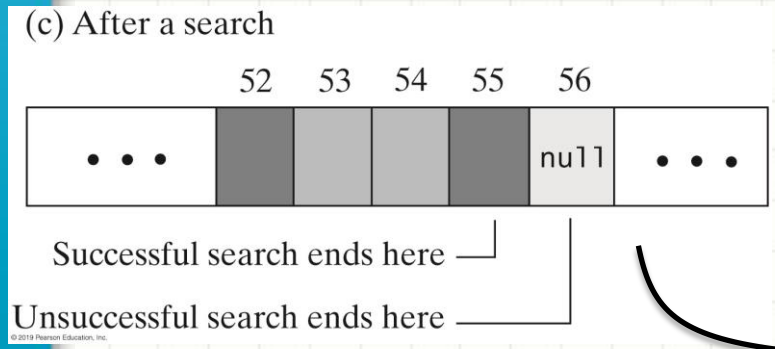
- Need to distinguish among three kinds of locations in the hash table
  - **`Occupied`**
    - location references an entry in the dictionary
  - **`Empty`**
    - location contains null and always has
  - **`Available`**
    - location's entry was removed from the dictionary

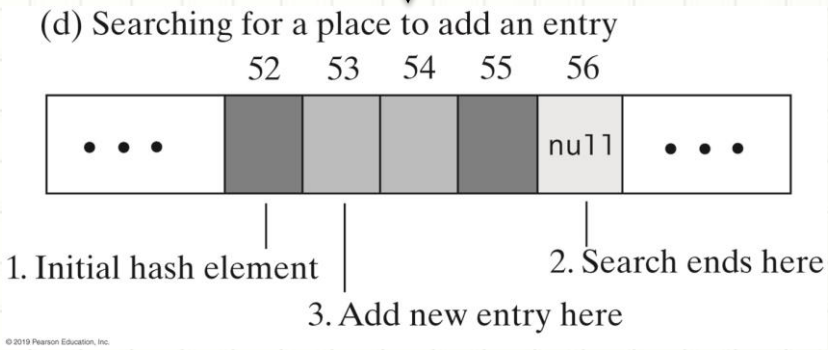
Linear Probing

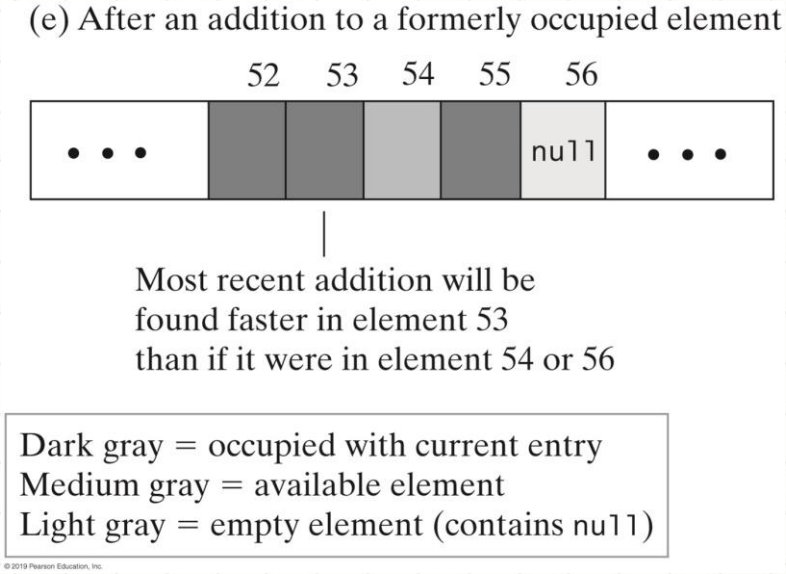
probe s











Linear Probing - Probe Algorithm

```
Algorithm probe(index, key)
// Searches the probe sequence that begins at index. Returns the index of either the element
// containing key or an available element in the hash table.
while (key is not found and hashTable[index] is not null)
{
        if (hashTable[index] references an entry in the dictionary)
        {
                if (the entry in hashTable[index] contains key)
                        Exit loop
                else
                        index = next probe index
        }
        else // hashTable[index] is available
        {
                if (this is the first available element encountered)
                        availableStateIndex = index
                index = next probe index
        }
}
if (key is found or an available element was not encountered)
        return index
else
        return availableStateIndex // Index of first entry removed
```

Linear Probe Algorithm

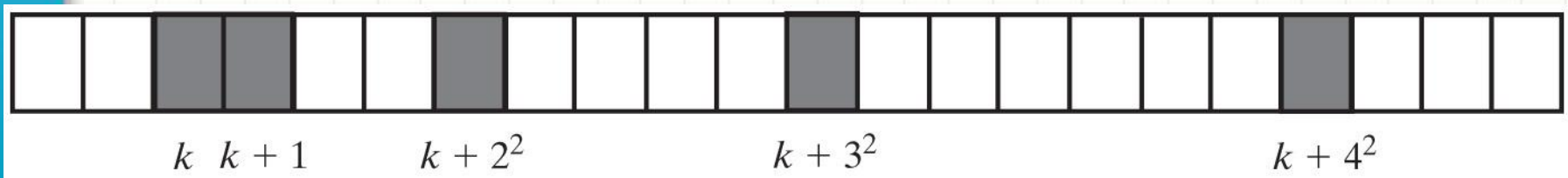
```
// Precondition: checkIntegrity has been called.
private int linearProbe(int index, K key)
{
        boolean found = false;
        int availableStateIndex = −1; // Index of first element in available state
        while ( !found && (hashTable[index] != null) )
        {
                if (hashTable[index] != AVAILABLE)
                {
                        if  (key.equals(hashTable[index].getKey()))
                                found = true; // Key found
                        else      // Follow probe  sequence
                                index = (index + 1) % hashTable.length; // Linear probing
                }
                else // Element in available state; skip it, but mark the first one encountered
                {
                        // Save index of first element in available state
                        if (availableStateIndex == −1) availableStateIndex = index;
                                index  =  (index  +  1) %  hashTable.length;     // Linear pro
                } // end if
        } // end while
        // Assertion: Either key or null is found at hashTable[index]
        if (found || (availableStateIndex == −1) )
                return index;                    // Index  of  either  key  or  null
        else
                return availableStateIndex; // Index of an available element
} // end  linearProbe
```

Clustering

- Collisions resolved with linear probing cause groups of consecutive locations in hash table to be occupied
  - Each group is called a ***cluster***
- Bigger clusters mean longer search times following collision

Open Addressing with Quadratic Probing

- Linear probing looks at consecutive locations beginning at index ***k***
- Quadratic probing:
  - Considers the locations at indices $k + j^2$
  - Uses the indices $k,\ k + 1,\ k + 4,\ k + 9,\ \ldots$



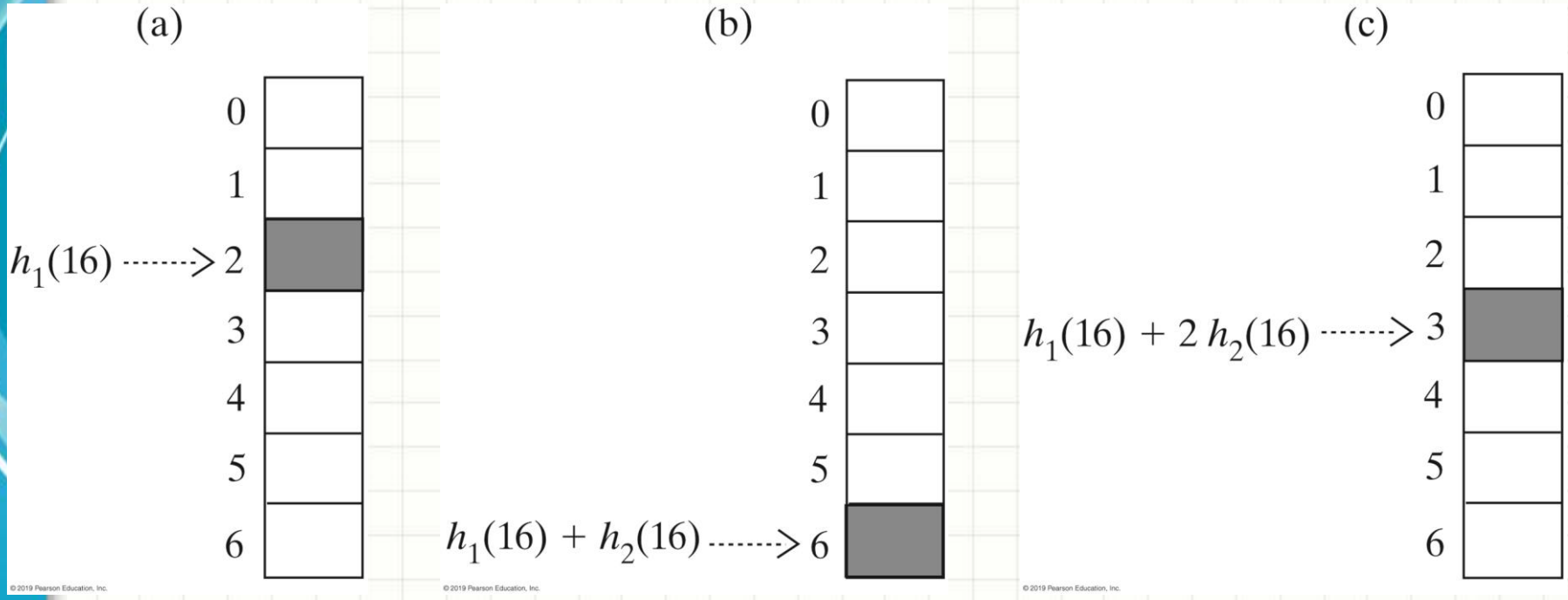
© 2019 Pearson Education, Inc.

A probe sequence of length five using quadratic probing

Open Addressing with Double Hashing

- Linear probing and quadratic probing add increments to k to define a probe sequence
  - Both are independent of the search key
- Double hashing uses a second hash function to compute these increments
  - This is a key-dependent method.

Open Addressing with Double Hashing



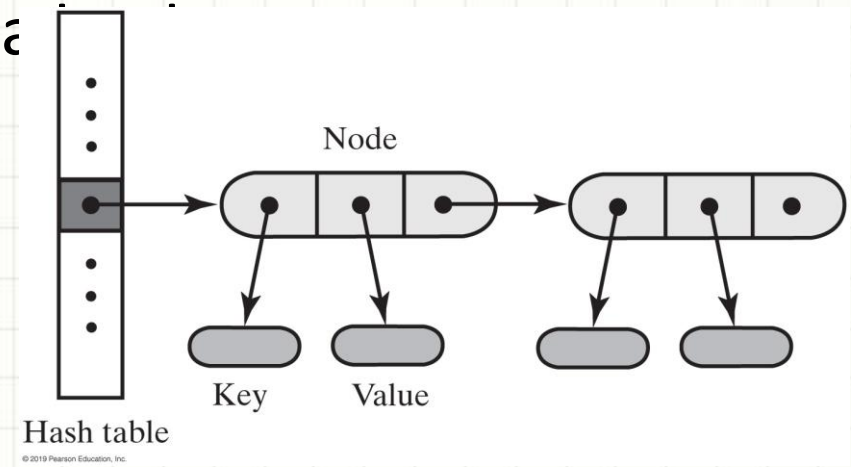
- The first three elements in a probe sequence generated by double hashing for the search key 16

Potential Problem with Open Addressing

- Recall each location is either occupied, empty, or available
  - Frequent additions and removals can result in no locations that are null
- Thus searching a probe sequence will not work
- Consider separate chaining as a solution

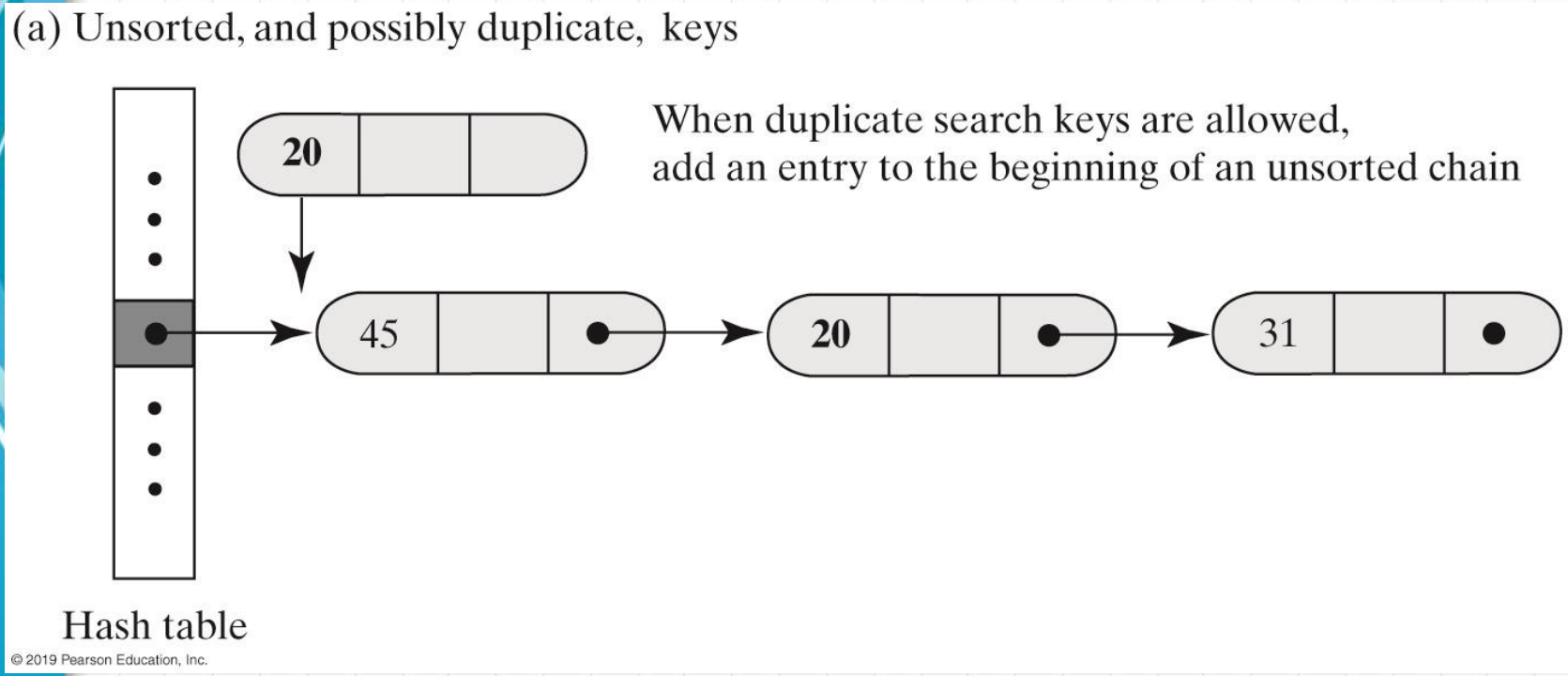
Separate Chaining

- Alter the structure of the hash table
  - Each location can represent more than one value.
  - Such a location is called a bucket
- Decide how to represent a
  - **`list, sorted list`**
  - **`array`**
  - **`linked nodes`**
  - **`vector`**



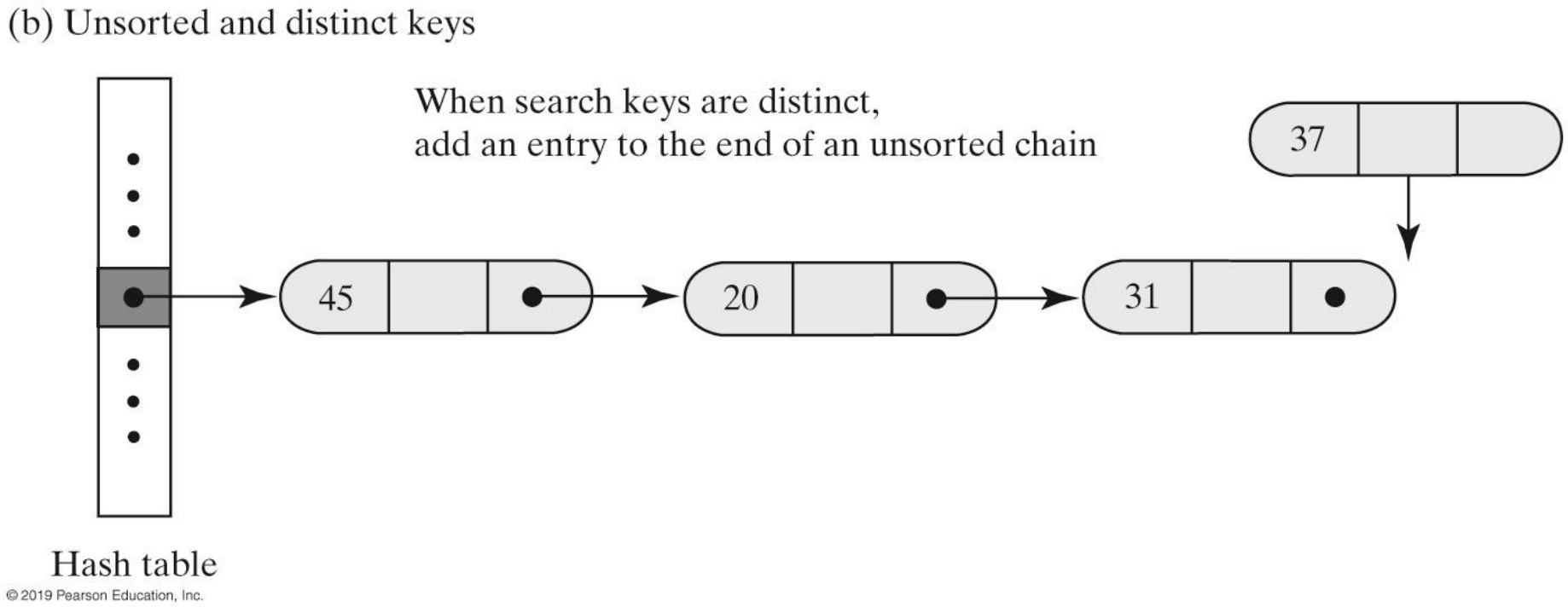
A hash table for use with separate chaining; each bucket is a chain of linked nodes

Separate Chaining



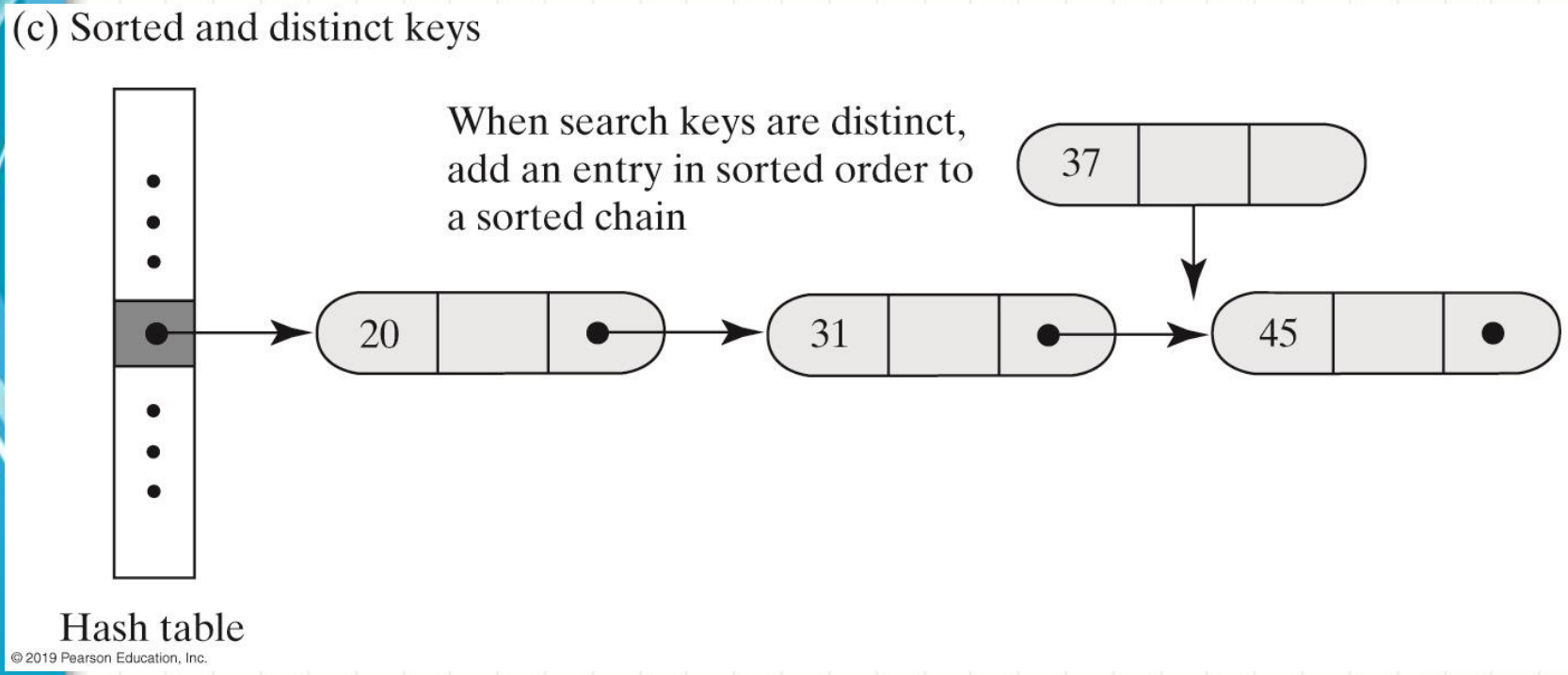
- Inserting a new entry into a linked bucket according to the nature of the integer search keys

Separate Chaining



- Inserting a new entry into a linked bucket according to the nature of the integer search keys

Separate Chaining



- Inserting a new entry into a linked bucket according to the nature of the integer search keys

Separate Chaining

- Algorithm for the dictionary's `add` method.

```
Algorithm add(key, value)
index = getHashIndex(key)
if (hashTable[index] == null)
{
        hashTable[index] = new Node(key, value)
        numberOfEntries++
        return null
}
else
{
    Search the chain that begins at hashTable[index] for a node that contains key
    if (key is found)
    { // Assume currentNode references the node that contains
            key oldValue = currentNode.getValue()
            currentNode.setValue(value)
            return oldValue
    }
    else // Add new node to end of chain
    {//Assume nodeBefore references the last node
            newNode = new Node(key, value)
            nodeBefore.setNextNode(newNode) numberOfEntries++
            return null
    }
}
```

Separate Chaining

- Algorithm for the dictionary's `remove` method.

```
Algorithm remove(key)
index = getHashIndex(key)
Search the chain that begins at hashTable[index] for a node that contains key
if (key is found)
{
        Remove the node that contains key from the chain
        numberOfEntries––
        return value in removed node
}
else
        return null
```

Separate Chaining

- Algorithm for the dictionary's `getValue` method.

Algorithm **getValue(key)**

index = getHashIndex(key)

Search the chain that begins at hashTable[index] *for a node that contains* key

if (key *is found*)

    return *value in found node*

else

    return null

Efficiency of Hashing

- Observations about the time efficiency of these operations
  - Successful retrieval/removal has same efficiency as successful search
  - Unsuccessful retrieval/removal has same efficiency as unsuccessful search
  - Successful addition has same efficiency as unsuccessful search
  - Unsuccessful addition has same efficiency as successful search

Load Factor

- Definition of load factor

$$\lambda = \frac{\textit{Number of entries in the dictionary}}{\textit{Number of locations in the hash table}}$$

  - Never negative
  - For open addressing, $1 \geq \lambda$
  - For separate chaining, λ has no maximum value
  - Restricting size of λ improves performance

Cost of Open Addressing

- Average number of searches for linear probing

For unsuccessful search:

$$\frac{1}{2}\left\{1 + \frac{1}{(1-\lambda)^2}\right\}$$

For successful search:

$$\frac{1}{2}\left\{1 + \frac{1}{(1-\lambda)}\right\}$$

Cost of Open Addressing

λ	[illegible]	[illegible]
0.1	1.1	1.1
0.3	1.5	1.2
0.5	2.5	1.5
0.7	6.1	2.2
0.9	50.5	5.5

- The average number of comparisons required by a search of the hash table for given values of the load factor λ when using linear probing

Quadratic Probing and Double Hashing

- Average number of com needed

  For unsuccessful search: $$\frac{1}{(1-\lambda)}$$

  For successful search: $$\frac{1}{\lambda}\log\left(\frac{1}{1-\lambda}\right)$$

Quadratic Probing and Double Hashing

λ	[illegible]	[illegible]
0.1	1.1	1.1
0.3	1.5	1.2
0.5	2.0	1.4
0.7	3.3	1.7
0.9	10.0	2.6

- The average number of comparisons required by a search of the hash table for given values of the load factor λ when using either quadratic probing or double hashing

Cost of Separate Chaining

- Average number of comparisons during a search when separate chaining is used:

For unsuccessful search: λ

For successful search: $1 + \lambda/2$

To maintain reasonable efficiency, you should keep $\lambda < 1$.

Cost of Separate Chaining

0.1	0.1	1.1
0.3	0.3	1.2
0.5	0.5	1.3
0.7	0.7	1.4
0.9	0.9	1.5
1.1	1.1	1.6
1.3	1.3	1.7
1.5	1.5	1.8
1.7	1.7	1.9
1.9	1.9	2.0
2.0	2.0	2.0

- The average number of comparisons required by a search of the hash table for given values of the load factor λ when using separate chaining

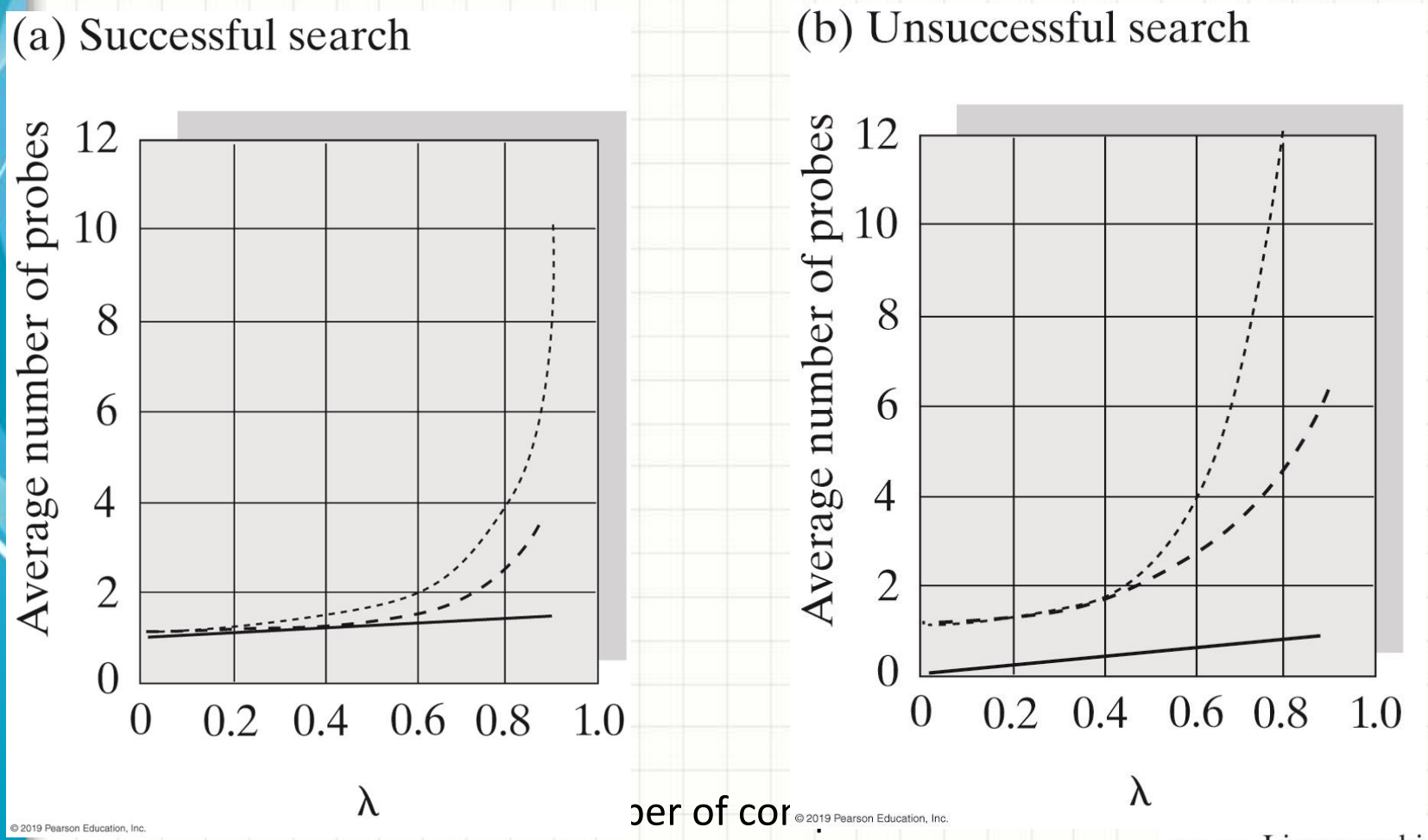
Maintaining the Performance of Hashing

- To maintain efficiency, restrict the size of λ as follows:
  - $\lambda < 0.5$ for open addressing
  - $\lambda < 1.0$ for separate chaining
- Should the load factor exceed these bounds
  - Increase the size of the hash table

Rehashing

- When the load factor λ becomes too large must resize the hash table
- Compute the table's new size
  - Double its present size
  - Increase the result to the next prime number
  - Use method add to add the current entries in dictionary to new hash table

Comparing Schemes for Collision Resolution



© 2019 Pearson Education, Inc.

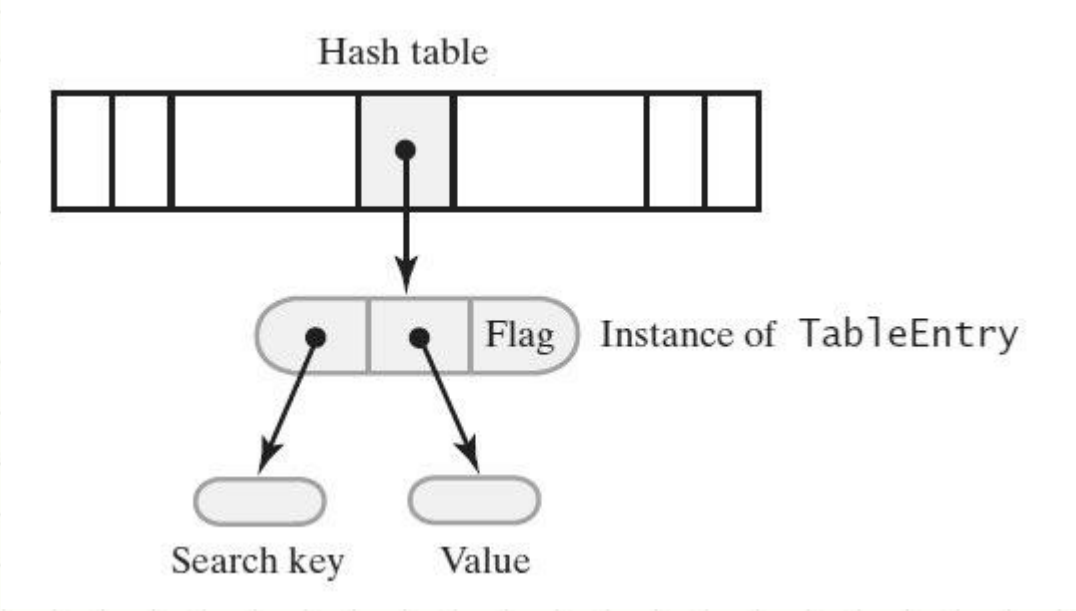
© 2019 Pearson Education, Inc.

ber of cor

required by a search of the hash table versus the load factor λ for four collision resolution techniques

Dictionary Implementation That Uses Hashing

- A hash table and one of its entry objects



Dictionary Implementation That Uses Hashing

- Private class `TableEntry`, make it internal to the dictionary class.

```
private static class TableEntry<S, T>
  {
  private S key;
  private T value;
  private States state;  // Flags whether this entry is in the hash table
  private enum States {CURRENT, REMOVED} // Possible values of state

  private TableEntry(S searchKey, T dataValue)
  {
    key = searchKey;
    value = dataValue;
    state = States.CURRENT;
  } // end constructor

                    // method implementations go here

  } // end TableEntry
```

A Dictionary that Uses Hashing

```
public class HashedDictionary<K, V> implements DictionaryInterface<K, V>
{
   // The dictionary:
      private int numberOfEntries;
      private static final int DEFAULT_CAPACITY = 5; // Must be prime
      private static final int MAX_CAPACITY = 10000;

   // The hash table:
      private Entry<K, V>[] hashTable;
   private int tableSize;                          // Must be prime
   private static final int MAX_SIZE = 2 * MAX_CAPACITY;
   private boolean integrityOK = false;
      private static final double MAX_LOAD_FACTOR = 0.5; // Fraction of
                                    // hash table that can be filled
      protected final Entry<K, V> AVAILABLE = new Entry<>(null, null);

      public HashedDictionary()
      {
      this(DEFAULT_CAPACITY); // Call next constructor
      } // end default constructor
```

A Dictionary that Uses Hashing

```
  public HashedDictionary(int initialCapacity)
  {
 initialCapacity = checkCapacity(initialCapacity);
  numberOfEntries = 0;    // Dictionary is empty

 // Set up hash table:
  // Initial size of hash table is same as initialCapacity if it is prime;
  // otherwise increase it until it is prime size
  int tableSize = getNextPrime(initialCapacity);
 checkSize(tableSize);   // Check that size is not too large

  // The cast is safe because the new array contains null entries
 @SuppressWarnings("unchecked")
 Entry<K, V>[] temp = (Entry<K, V>[])new Entry[tableSize];
 hashTable = temp;
 integrityOK = true;
  } // end constructor
/* Implementations of methods in DictionaryInterface are here. . . .

  Implementations of private methods are here.  . . . */

  protected final class Entry<S, T>
  {
    /*       . . . */
  } // end Entry
} // end HashedDictionary
```

- An outline of the class `HashedDictionary`

Required Methods

- Algorithm for the retrieval method `getValue`

```
Algorithm getValue(key)
// Returns the value associated with the given search key, if it is in the dictionary.
// Otherwise, returns null.
if (key is found)
   return value in found entry
else
   return null
```

Required Methods

- Implementation of method `getValue`

```
public V getValue(K key)
{
   checkIntegrity();
   V result = null;

   int index = getHashIndex(key);
   if ((hashTable[index] != null) && (hashTable[index] != AVAILABLE))
      result = hashTable[index].getValue(); // Key found; get value
   // Else key not found; return null

   return result;
} // end getValue
```

Required Methods

- Pseudocode for method `remove`

```
Algorithm remove(key)
// Removes a specific entry from the dictionary, given its search key.
// Returns either the value that was associated with the search key or null if no such object
// exists.
removedValue = null
index = getHashIndex(key)
if (key is found) // hashTable[index] is not null and does not equal AVAILABLE
{
        removedValue = hashTable[index].getValue()
        hashTable[index] = AVAILABLE
        numberOfEntries--
}
return removedValue
```

Required Methods

- Algorithm for adding a new entry

```
Algorithm add(key, value)
// Adds a new key-value entry to the dictionary. If key is already in the dictionary,
// returns its corresponding value and replaces it in the dictionary with value.
if ((key == null) or (value == null))
        Throw an exception
index = getHashIndex(key)
if (key is not found)
{ // Add entry to hash table
        hashTable[index] = new Entry(key, value)
        numberOfEntries++
        oldValue = null
}
else // Search key is in table; replace and return entry's value
{
        oldValue = hashTable[index].getValue()
        hashTable[index].setValue(value)
}
// Ensure that hash table is large enough for another addition
if (hash table is too full)
        Enlarge hash table
return oldValue
```

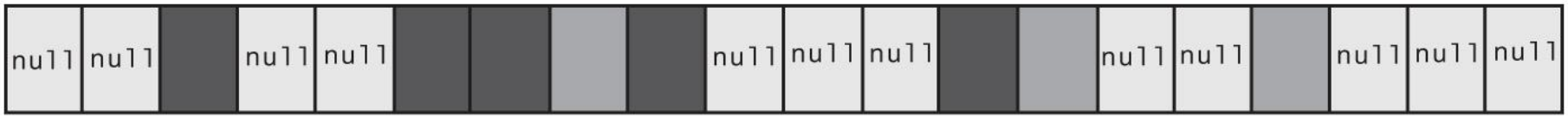
Required Methods

- Private method `enlargeHashTable`

```
private void enlargeHashTable()
{
   Entry<K, V>[] oldTable = hashTable;
   int oldSize = hashTable.length;
   int newSize = getNextPrime(oldSize + oldSize);
   checkSize(newSize);

   // The cast is safe because the new array contains null entries
   @SuppressWarnings("unchecked")
   Entry<K, V>[] temp = (Entry<K, V>[])new Entry[newSize];
   hashTable = temp;
   numberOfEntries = 0; // Reset number of dictionary entries, since
                        // it will be incremented by add during rehash
   // Rehash dictionary entries from old array to the new and bigger array;
   // skip elements that contain null or AVAILABLE
   for (int index = 0; index < oldSize; index++)
   {
      if ( (oldTable[index] != null) && oldTable[index] != AVAILABLE )
         add(oldTable[index].getKey(), oldTable[index].getValue());
   } // end for
} // end enlargeHashTable
```

Hash Tables and Iterators



Dark gray = occupied with current entry
Medium gray = available location
Light gray = empty location (null)

© 2019 Pearson Education, Inc.

- A hash table containing occupied elements, available elements, and null values

Hash Tables and Iterators (Part 1)

- Implementation of `KeyIterator`

```
private class KeyIterator implements Iterator<K>
{
   private int currentIndex; // Current position in hash table
   private int numberLeft;   // Number of entries left in iteration

   private KeyIterator()
   {
      currentIndex = 0;
      numberLeft = numberOfEntries;
   } // end default constructor

   public boolean hasNext()
   {
      return numberLeft > 0;
   } // end hasNext

   public void remove()
   {
      throw new UnsupportedOperationException();
   } // end remove
```

Hash Tables and Iterators (Part 2)

```
public K next()
 {
    K result = null;

    if (hasNext())
    {
       // Skip table locations that do not contain a current entry
       while ( (hashTable[currentIndex] == null) ||
                                                (hashTable[currentIndex] == AVAILABLE) )
       {
          currentIndex++;
       } // end while

       result = hashTable[currentIndex].getKey();
       numberLeft--;
       currentIndex++;
    }
    else
       throw new NoSuchElementException();

    return result;
 } // end next
} // end KeyIterator
```

- Implementation of `KeyIterator`

Java Class Library: The Class `HashMap`

- Hash table is a collection of buckets
  - Each bucket contains several entries
- Variety of constructors provided
- Default maximum load factor of 0.75
  - When limit exceeded, size of table increased by rehashing
- Possible to avoid rehashing by setting number of buckets initially larger

Java Class Library: The Class `HashSet`

- Implements the interface **`java.util.Set`**
- **`HashSet`** uses an instance of the class **`HashMap`**
- Variety of constructors provided in class